

Jetpack Compose第四章第二节教学讲义（手势与滚动交互）

各位同学，前面我们学过Compose的UI布局和主题样式，做出的界面都是“静态”的——用户点击、滑动时没有任何反应。而优秀的APP必然离不开“动态交互”，比如点击按钮提交表单、滑动列表查看商品、拖动图片调整位置。这节课咱们就聚焦Compose的手势与滚动能力，从最基础的点击到复杂的多点触控，一步步让你的组件“活”起来！

一、核心认知：Compose的手势体系

在Compose中，手势交互主要通过**修饰符（Modifier）**实现——就像给组件“贴一个交互能力标签”，比如给Text组件加 `clickable` 修饰符，它就变成了“可点击的文本”。

Compose的手势体系按复杂度分为三层，咱们从浅到深逐步学习：

1. **基础手势**：点击、长按（最常用，直接通过现成修饰符实现）；
2. **滚动与滑动**：滚动列表、滑动删除（基于滚动修饰符，处理连续位移）；
3. **进阶触控**：拖动、多点触控（处理复杂的手指位置变化）。

二、基础手势：点击与长按（最常用）

点击（Click）和长按（Long Click）是APP中最基础的交互，比如按钮点击、列表项长按弹出菜单。Compose直接提供了 `clickable` 修饰符，开箱即用。

1. 简单点击：clickable修饰符

给任意组件添加 `clickable` 修饰符，就能监听点击事件，相当于给组件加了“点击响应”的能力。

实战1：可点击的文本按钮

Code block

```
1 import androidx.compose.foundation.clickable
2 import androidx.compose.foundation.layout.padding
3 import androidx.compose.material3.Text
4 import androidx.compose.runtime.Composable
5 import androidx.compose.runtime.getValue
6 import androidx.compose.runtime.mutableStateOf
7 import androidx.compose.runtime.remember
8 import androidx.compose.runtime.setValue
```

```

9  import androidx.compose.ui.Modifier
10 import androidx.compose.ui.unit.dp
11
12 @Composable
13 fun ClickableTextDemo() {
14     // 状态：记录点击次数（用remember保存状态，防止重组丢失）
15     var clickCount by remember { mutableStateOf(0) }
16
17     Text(
18         text = "点击我（已点击$clickCount次）",
19         modifier = Modifier
20             .padding(16.dp)
21             // 核心：添加clickable修饰符，监听点击
22             .clickable {
23                 // 点击时执行的逻辑
24                 clickCount++
25             }
26     )
27 }

```

核心知识点

- `remember`：用于保存组件重组时不会丢失的状态（比如点击次数），没有它点击后数字会立刻恢复为0；
- `mutableStateOf`：创建可观察的状态，当状态变化时，Compose会自动重组使用该状态的组件（比如Text会重新显示最新的点击次数）；
- `clickable` 的 lambda 表达式：点击发生时执行的逻辑，这里我们让点击次数加1。

2. 点击+长按：同时监听两种手势

如果需要同时监听点击和长按，可以给 `clickable` 传入 `onLongClick` 参数，实现“短按触发点击，长按触发另一个逻辑”。

实战2：支持长按的计数器

Code block

```

1  import androidx.compose.foundation.clickable
2  import androidx.compose.foundation.layout.padding
3  import androidx.compose.material3.Text
4  import androidx.compose.runtime.Composable
5  import androidx.compose.runtime.getValue
6  import androidx.compose.runtime.mutableStateOf
7  import androidx.compose.runtime.remember
8  import androidx.compose.runtime.setValue

```

```

9  import androidx.compose.ui.Modifier
10 import androidx.compose.ui.unit.dp
11
12 @Composable
13 fun ClickAndLongClickDemo() {
14     var count by remember { mutableStateOf(0) }
15
16     Text(
17         text = "点击+1, 长按重置 (当前$count) ",
18         modifier = Modifier
19             .padding(16.dp)
20             .clickable(
21                 onClick = { count++ }, // 点击逻辑: 加1
22                 onLongClick = { count = 0 } // 长按逻辑: 重置为0
23             )
24     )
25 }

```

效果说明

短按文本时，计数加1；长按文本（按住不放约1秒）时，计数重置为0——这就是多手势监听的核心用法，常见于“列表项长按删除”等场景。

三、滚动交互：处理“内容超出屏幕”的情况

当组件内容超出屏幕范围时（比如长文本、多列表项），就需要添加滚动能力，让用户能滑动查看全部内容。Compose提供了两种核心滚动修饰符：`verticalScroll`（垂直滚动）和 `horizontalScroll`（水平滚动）。

1. 基础滚动：verticalScroll与horizontalScroll

这两个修饰符直接作用于容器组件（如Column、Row），让容器内的内容支持滚动，用法完全一致，仅方向不同。

实战3：垂直滚动的长文本

Code block

```

1  import androidx.compose.foundation.layout.Column
2  import androidx.compose.foundation.layout.padding
3  import androidx.compose.foundation.verticalScroll
4  import androidx.compose.material3.Text
5  import androidx.compose.runtime.Composable
6  import androidx.compose.ui.Modifier
7  import androidx.compose.ui.unit.dp

```

```

8
9 @Composable
10 fun VerticalScrollTextDemo() {
11     // 长文本（模拟内容超出屏幕）
12     val longText = """
13         这是一段超长的文本，用于演示垂直滚动效果。
14         当文本内容的高度超过屏幕高度时，
15         给Column添加verticalScroll修饰符，
16         用户就能通过上下滑动查看全部内容。
17         滚动修饰符的核心作用是：
18         让容器内超出可视范围的内容，
19         可以通过滑动交互展示出来。
20         这在展示长文章、详细说明时非常常用。
21         再加点内容凑长度...
22         1. 滚动修饰符不改变组件大小，只提供滚动能力
23         2. 必须作用于Column/Row等容器组件
24         3. 垂直滚动用verticalScroll，水平用horizontalScroll
25     """.trimIndent()
26
27     Column(
28         modifier = Modifier
29             .padding(16.dp)
30             // 核心：添加垂直滚动修饰符
31             .verticalScroll(
32                 // rememberScrollState(): 保存滚动位置，防止重组后回到顶部
33                 state = androidx.compose.foundation.rememberScrollState()
34             )
35     ) {
36         Text(text = longText)
37     }
38 }

```

实战4：水平滚动的横向列表

Code block

```

1  import androidx.compose.foundation.layout.Row
2  import androidx.compose.foundation.layout.size
3  import androidx.compose.foundation.horizontalScroll
4  import androidx.compose.material3.Card
5  import androidx.compose.material3.Text
6  import androidx.compose.runtime.Composable
7  import androidx.compose.ui.Modifier
8  import androidx.compose.ui.unit.dp
9
10 @Composable

```

```

11 fun HorizontalScrollRowDemo() {
12     Row(
13         modifier = Modifier
14             .padding(16.dp)
15             // 核心：水平滚动修饰符
16             .horizontalScroll(
17                 state = androidx.compose.foundation.rememberScrollState()
18             )
19     ) {
20         // 多个卡片，宽度总和超过屏幕，支持水平滑动
21         repeat(5) { index ->
22             Card(
23                 modifier = Modifier
24                     .size(width = 150.dp, height = 100.dp)
25                     .padding(end = 8.dp)
26             ) {
27                 Text(
28                     text = "横向卡片$index",
29                     modifier = Modifier.padding(16.dp)
30                 )
31             }
32         }
33     }
34 }

```

核心知识点

- `rememberScrollState()`：用于保存滚动位置——如果没有它，每次组件重组（比如屏幕旋转）后，滚动位置会重置到顶部/最左侧；
- 滚动修饰符**必须作用于容器组件**（如`Column`、`Row`），不能直接作用于`Text`等子组件；
- 垂直滚动用 `verticalScroll`，水平滚动用 `horizontalScroll`，用法完全对称。

2. 可滚动组件：LazyColumn与LazyRow（性能优化）

前面的 `verticalScroll` 会“一次性加载所有内容”——如果列表有1000个item，即使只显示10个，也会全部创建，严重影响性能。这时候就需要**懒加载滚动组件**：LazyColumn（垂直懒加载列表）和LazyRow（水平懒加载列表）。

核心优势：只加载当前屏幕可见的item，滑动时销毁不可见的item，大幅提升性能，是列表类场景的首选。

实战5：LazyColumn懒加载列表

Code block

```
1  import androidx.compose.foundation.layout.padding
2  import androidx.compose.material3.Card
3  import androidx.compose.material3.Text
4  import androidx.compose.runtime.Composable
5  import androidx.compose.ui.Modifier
6  import androidx.compose.ui.unit.dp
7
8  @Composable
9  fun LazyColumnDemo() {
10     // 模拟100个列表项
11     val itemList = List(100) { "列表项 $it" }
12
13     // 懒加载垂直列表
14     androidx.compose.foundation.lazy.LazyColumn(
15         modifier = Modifier.padding(16.dp),
16         // 列表项之间的间距
17         verticalArrangement =
18         androidx.compose.foundation.layout.Arrangement.spacedBy(8.dp)
19     ) {
20         // 遍历列表，创建列表项（只加载可见项）
21         items(itemList) { itemText ->
22             Card(
23                 modifier = Modifier.fillMaxWidth()
24             ) {
25                 Text(
26                     text = itemText,
27                     modifier = Modifier.padding(16.dp)
28                 )
29             }
30         }
31     }
```

关键对比：LazyColumn vs Column+verticalScroll

对比维度	Column + verticalScroll	LazyColumn
加载方式	一次性加载所有item	只加载屏幕可见item
性能	item数量多时性能差（卡顿、内存占用高）	性能优异，适合大量item（如1000+）
适用场景	少量内容（如长文本、10个以内item）	大量列表项（如商品列表、消息列表）
核心API		items() 方法遍历列表

```
verticalScroll(rememberScrollState())
```

四、进阶交互：拖动与滑动

拖动（Drag）和滑动（Swipe）是更复杂的手势——拖动是“按住组件并移动”（如拖动图片调整位置），滑动是“快速划过组件”（如滑动删除列表项）。Compose通过 `pointerInput` 修饰符处理这些底层手势。

1. 拖动：DragGestureDetector

实现组件拖动需要监听“手指按下→移动→抬起”的完整流程，Compose提供了 `detectDragGestures` 手势检测器，封装了这个流程。

实战6：可拖动的彩色方块

Code block

```
1  import androidx.compose.foundation.background
2  import androidx.compose.foundation.gestures.detectDragGestures
3  import androidx.compose.foundation.layout.Box
4  import androidx.compose.foundation.layout.fillMaxSize
5  import androidx.compose.foundation.layout.size
6  import androidx.compose.runtime.Composable
7  import androidx.compose.runtime.getValue
8  import androidx.compose.runtime.mutableStateOf
9  import androidx.compose.runtime.remember
10 import androidx.compose.runtime.setValue
11 import androidx.compose.ui.Modifier
12 import androidx.compose.ui.graphics.Color
13 import androidx.compose.ui.input.pointer.pointerInput
14 import androidx.compose.ui.unit.dp
15
16 @Composable
17 fun DraggableBoxDemo() {
18     // 状态：记录方块的位置（x轴和y轴偏移量）
19     var offsetX by remember { mutableStateOf(0f) }
20     var offsetY by remember { mutableStateOf(0f) }
21
22     Box(
23         modifier = Modifier.fillMaxSize()
24     ) {
25         Box(
26             modifier = Modifier
27                 .size(100.dp)
28                 .background(Color.Blue)
```

```

29         // 核心：pointerInput修饰符处理手势
30         .pointerInput(Unit) {
31             // 检测拖动手势
32             detectDragGestures(
33                 // 拖动过程中触发：delta是手指移动的距离
34                 onDrag = { change, delta ->
35                     // 更新偏移量（累加手指移动距离）
36                     offsetX += delta.x
37                     offsetY += delta.y
38                     // 标记手势已处理
39                     change.consume()
40                 }
41             )
42         }
43         // 通过offset修饰符设置方块位置
44         .offset(
45             x = offsetX.dp,
46             y = offsetY.dp
47         )
48     )
49 }
50 }

```

核心知识点

- `pointerInput`：处理底层手势的入口修饰符，参数通常传 `Unit`（表示手势逻辑不依赖外部状态）；
- `detectDragGestures`：拖动手势检测器，`onDrag` 参数接收“手指移动时的回调”；
- `delta`：手指在本次移动中的偏移量（x是水平方向，y是垂直方向），通过累加delta得到方块的总偏移量；
- `offset` 修饰符：根据偏移量调整组件的位置，实现“拖动跟随手指”的效果。

2. 滑动：SwipeToDismiss（滑动删除）

滑动删除是列表常见交互（如微信消息左滑删除），Compose提供了 `SwipeToDismiss` 组件，无需手动处理底层手势，直接实现滑动删除效果。

实战7：支持滑动删除的LazyColumn

Code block

```

1  import androidx.compose.foundation.background
2  import androidx.compose.foundation.layout.Box
3  import androidx.compose.foundation.layout.fillMaxSize

```



```

4  import androidx.compose.foundation.layout.fillMaxWidth
5  import androidx.compose.foundation.layout.padding
6  import androidx.compose.foundation.lazy.LazyColumn
7  import androidx.compose.foundation.lazy.items
8  import androidx.compose.material.icons.Icons
9  import androidx.compose.material.icons.filled.Delete
10 import androidx.compose.material3.Card
11 import androidx.compose.material3.Icon
12 import androidx.compose.material3.Text
13 import androidx.compose.runtime.Composable
14 import androidx.compose.runtime.getValue
15 import androidx.compose.runtime.mutableStateListOf
16 import androidx.compose.runtime.remember
17 import androidx.compose.ui.Alignment
18 import androidx.compose.ui.Modifier
19 import androidx.compose.ui.graphics.Color
20 import androidx.compose.ui.unit.dp
21 import androidx.compose.material3.SwipeToDismiss
22 import androidx.compose.material3.rememberDismissState
23 import androidx.compose.material3.DismissDirection
24 import androidx.compose.material3.DismissValue
25
26 @Composable
27 fun SwipeToDeleteDemo() {
28     // 可变列表：支持添加/删除元素（mutableStateListOf是可观察的列表）
29     val itemList = remember {
30         mutableStateListOf("消息1", "消息2", "消息3", "消息4")
31     }
32
33     LazyColumn(
34         modifier = Modifier.fillMaxSize(),
35         verticalArrangement =
36             androidx.compose.foundation.layout.Arrangement.spacedBy(8.dp)
37     ) {
38         items(
39             items = itemList,
40             // 给每个item设置唯一key，确保滑动删除时动画流畅
41             key = { it }
42         ) { item ->
43             // 1. 记住滑动状态
44             val dismissState = rememberDismissState()
45
46             // 2. 滑动删除组件
47             SwipeToDismiss(
48                 state = dismissState,
49                 // 允许向左滑动删除
50                 directions = setOf(DdismissDirection.StartToEnd),

```

```

50         // 滑动时显示的“背景”（比如删除图标）
51         background = {
52             Box(
53                 modifier = Modifier
54                     .fillMaxWidth()
55                     .background(Color.Red)
56                     .padding(16.dp),
57                 contentAlignment = Alignment.CenterStart
58             ) {
59                 Icon(
60                     imageVector = Icons.Default.Delete,
61                     contentDescription = "删除",
62                     tint = Color.White
63                 )
64             }
65         },
66         // 滑动的“目标组件”（列表项）
67         dismissContent = {
68             Card(modifier = Modifier.fillMaxWidth()) {
69                 Text(
70                     text = item,
71                     modifier = Modifier.padding(16.dp)
72                 )
73             }
74         }
75     )
76
77     // 3. 监听滑动状态：当滑动完成后删除item
78     if (dismissState.isDismissed(DismissDirection.StartToEnd)) {
79         // 从列表中删除当前item
80         itemList.remove(item)
81         // 重置滑动状态（否则下次进入会残留状态）
82         dismissState.reset()
83     }
84 }
85 }
86 }

```

核心知识点

- `mutableStateListOf`：可观察的列表，当列表元素添加/删除时，`LazyColumn`会自动重组更新UI；
- `rememberDismissState`：保存滑动删除的状态（如是否滑动完成）；
- `background`：滑动时显示在列表项下方的背景（这里用红色背景+删除图标，提示用户“滑动可删除”）；

- `dismissContent`：需要添加滑动删除能力的目标组件（即列表项Card）；
- `isDismissed`：判断用户是否完成滑动操作，完成后从列表中删除对应的item。

五、高阶能力：多点触控（捏合缩放）

多点触控是指“多个手指同时操作”，最常见的场景是“捏合缩放图片”。Compose通过 `detectTransformGestures` 手势检测器实现，它能同时监听“拖动”和“缩放”手势。

实战8：支持捏合缩放的图片

Code block

```
1  import androidx.compose.foundation.Image
2  import androidx.compose.foundation.gestures.detectTransformGestures
3  import androidx.compose.foundation.layout.Box
4  import androidx.compose.foundation.layout.fillMaxSize
5  import androidx.compose.foundation.layout.size
6  import androidx.compose.runtime.Composable
7  import androidx.compose.runtime.getValue
8  import androidx.compose.runtime.mutableStateOf
9  import androidx.compose.runtime.remember
10 import androidx.compose.runtime.setValue
11 import androidx.compose.ui.Alignment
12 import androidx.compose.ui.Modifier
13 import androidx.compose.ui.input.pointer.pointerInput
14 import androidx.compose.ui.layout.ContentScale
15 import androidx.compose.ui.res.painterResource
16 import androidx.compose.ui.unit.dp
17 import com.example.yourapp.R // 替换为你的图片资源ID
18
19 @Composable
20 fun PinchToZoomImageDemo() {
21     // 状态1: 缩放比例（默认1f，即原始大小）
22     var scale by remember { mutableStateOf(1f) }
23     // 状态2: 旋转角度（默认0f，可选）
24     var rotation by remember { mutableStateOf(0f) }
25     // 状态3: 拖动偏移量
26     var offsetX by remember { mutableStateOf(0f) }
27     var offsetY by remember { mutableStateOf(0f) }
28
29     Box(
30         modifier = Modifier.fillMaxSize(),
31         contentAlignment = Alignment.Center
32     ) {
33         Image(
34             painter = painterResource(id = R.drawable.ic_demo), // 替换为你的图片
```

```

35         contentDescription = "可缩放图片",
36         contentScale = ContentScale.Fit,
37         modifier = Modifier
38             .size(300.dp)
39             // 处理多点触控手势
40             .pointerInput(Unit) {
41                 detectTransformGestures(
42                     onGesture = { centroid, pan, zoom, rotationChange ->
43                         // 1. 更新缩放比例（限制在0.5f~3f之间，避免缩太小或太大）
44                         scale = (scale * zoom).coerceIn(0.5f, 3f)
45                         // 2. 更新旋转角度
46                         rotation += rotationChange
47                         // 3. 更新拖动偏移量（结合缩放比例，让拖动更自然）
48                         offsetX += pan.x * scale
49                         offsetY += pan.y * scale
50                     }
51                 )
52             }
53         // 应用缩放、旋转、偏移
54         .graphicsLayer(
55             scaleX = scale,
56             scaleY = scale,
57             rotationZ = rotation,
58             translationX = offsetX,
59             translationY = offsetY
60         )
61     )
62 }
63 }

```

核心知识点

- `detectTransformGestures`：多点触控手势检测器，`onGesture` 参数接收四个关键参数：
 - `zoom`：缩放比例变化（大于1是放大，小于1是缩小）；
 - `pan`：拖动偏移量（和前面的拖动delta类似）；
 - `rotationChange`：旋转角度变化（用于实现旋转功能）；
- `coerceIn(0.5f, 3f)`：限制缩放比例的范围，避免图片缩到看不见或放得过大；
- `graphicsLayer`：用于应用缩放、旋转、偏移等图形变换，是实现复杂UI效果的核心修饰符。

六、嵌套滚动：解决“滚动冲突”

当两个滚动组件嵌套时（比如“可滚动的列表”放在“可滚动的页面”中），会出现“滚动冲突”——用户滑动时，系统不知道该让哪个组件响应。Compose通过 `nestedScroll` 修饰符解决这个问题，实现“父组件和子组件协同滚动”。

实战9：嵌套滚动的示例（CollapsingToolbar效果）

最常见的嵌套滚动场景是“折叠式标题栏”——滑动列表时，标题栏随列表滚动逐渐折叠，代码如下：

Code block

```
1  import androidx.compose.foundation.ScrollState
2  import androidx.compose.foundation.layout.Box
3  import androidx.compose.foundation.layout.Column
4  import androidx.compose.foundation.layout.fillMaxWidth
5  import androidx.compose.foundation.layout.height
6  import androidx.compose.foundation.layout.verticalScroll
7  import androidx.compose.material3.Text
8  import androidx.compose.runtime.Composable
9  import androidx.compose.ui.Modifier
10 import androidx.compose.ui.graphics.Color
11 import androidx.compose.ui.unit.dp
12
13 @Composable
14 fun NestedScrollDemo() {
15     // 父滚动状态
16     val parentScrollState = ScrollState(0)
17
18     Column {
19         // 1. 可折叠的标题栏（高度随滚动变化）
20         Box(
21             modifier = Modifier
22                 .fillMaxWidth()
23                 // 标题栏高度：滚动时从200dp缩小到56dp
24                 .height(200.dp -
parentScrollState.value.toFloat().coerceAtMost(144f))
25                 .background(Color.Blue)
26         ) {
27             Text(
28                 text = "折叠标题栏",
29                 color = Color.White,
30                 modifier = Modifier.align(androidx.compose.ui.Alignment.Center)
31             )
32         }
33
34         // 2. 子滚动列表（和父滚动协同）
35         Box(
36             modifier = Modifier
```

```

37         .weight(1f)
38         // 核心：嵌套滚动修饰符，关联父滚动状态
39         .nestedScroll(
40             connection = object {
41                 androidx.compose.foundation.gestures.NestedScrollConnection {
42                     // 子组件滚动前，先让父组件滚动
43                     override fun onPreScroll(available:
44                         androidx.compose.foundation.gestures.NestedScrollSource, source:
45                         androidx.compose.foundation.gestures.NestedScrollSource):
46                         androidx.compose.foundation.gestures.NestedScrollSource {
47                         // 计算父组件能滚动的距离
48                         val delta = available.y
49                         val newOffset = parentScrollState.value +
50                         delta.toInt()
51                         // 父组件滚动到极限前，优先滚动父组件
52                         if (newOffset in 0..144) {
53                             parentScrollState.scrollTo(newOffset)
54                             return available.copy(y = 0f) // 父组件消耗所有滚动距离
55                         }
56                         return available // 父组件无法滚动，让子组件滚动
57                     }
58                 }
59             )
60             .verticalScroll(rememberScrollState())
61         ) {
62             // 长列表内容
63             repeat(50) {
64                 Text(
65                     text = "嵌套滚动列表项 $it",
66                     modifier = Modifier
67                         .fillMaxWidth()
68                         .height(50.dp)
69                 )
70             }
71         }
72     }
73 }

```

核心逻辑

通过 `NestedScrollConnection` 的 `onPreScroll` 方法，定义“滚动事件的分发规则”：

1. 用户向上滑动时，先让父组件（标题栏）滚动（高度缩小）；
2. 当标题栏缩小到最小高度（56dp）后，父组件无法继续滚动，此时让子组件（列表）开始滚动；
3. 用户向下滑动时，先让子组件滚动到顶部，再让父组件滚动（标题栏高度恢复）。

七、小节回顾：手势与滚动核心API速查表

交互类型	核心API	适用场景
点击/长按	Modifier.clickable(onClick, onLongClick)	按钮、文本、列表项的基础交互
基础滚动	verticalScroll/horizontalScroll + rememberScrollState()	少量内容（长文本、短列表）
懒加载滚动	LazyColumn/LazyRow + items()	大量列表项（商品列表、消息列表）
拖动	Modifier.pointerInput + detectDragGestures	拖动图片、滑块等组件
滑动删除	SwipeToDismiss + rememberDismissState()	列表项滑动删除
多点触控	Modifier.pointerInput + detectTransformGestures	图片捏合缩放、旋转
嵌套滚动	Modifier.nestedScroll + NestedScrollConnection	滚动组件嵌套（折叠标题栏）

开发避坑指南

1. 列表优先用LazyColumn/LazyRow：即使item数量不多，也建议用懒加载组件，避免后续扩展时性能问题；
2. 状态必须用remember保存：手势交互中的状态（如点击次数、滚动位置），必须用remember或rememberScrollState保存，否则重组后状态丢失；
3. 限制缩放/拖动范围：实现缩放、拖动时，用coerceIn限制范围，避免组件超出屏幕或变得不可用；
4. 滑动删除加key：LazyColumn的items必须传key，否则滑动删除时可能出现动画异常或数据错乱。

八、课后作业：动手实现“图片查看器”

需求：基于本节课知识点，实现一个支持“拖动、捏合缩放、滑动切换”的图片查看器，功能如下：

1. 基础功能：

展示多张图片（用LazyRow实现横向滑动切换）；
2. 单张图片支持捏合缩放（缩放范围0.5f~3f）；

3. 缩放后支持拖动图片查看局部；

4. 进阶功能：

 图片缩放后，横向滑动切换图片时，自动重置当前图片的缩放比例和位置；

5. 添加“当前图片索引/总张数”的文本提示（如“1/5”）；

6. 技术要求：

 用LazyRow实现图片横向切换；

7. 用detectTransformGestures实现捏合缩放和拖动；

8. 用mutableStateOf/mutableStateListOf管理状态；

9. 避免硬编码，状态变化时UI自动更新。

提示：图片资源可用drawable目录下的示例图片，或通过网络图片库加载。重点练习“多点触控”与“懒加载滚动”的结合，以及状态的联动管理（如切换图片时重置缩放状态），确保理解复杂手势交互中的状态同步逻辑。

（注：文档部分内容可能由 AI 生成）